

Guía de Laboratorio · Sesión 4

Del desastre a la resiliencia — volúmenes, respaldo y restauración probada

Curso 101-EAN · Gestión de Contenedores en Entornos de Nube Híbrida · Duración: 1 hora Plataforma: **Play with Docker** (navegador, gratis)

Hoy pierdes datos a propósito — dos veces. La primera te duele; la segunda no, porque ya tendrás volúmenes y un respaldo PROBADO. Entregable: la evidencia del ciclo completo y tu RTO cronometrado.

0. Preparación (5 min)

labs.play-with-docker.com → Login → Start → + **ADD NEW INSTANCE**. Crea la red de una vez:

```
docker network create ean-net
```

Parte 1 · El experimento cruel + el fix (15 min)

1.1 — Primero, siente la pérdida. Un contenedor SIN volumen:

```
docker run -d --name efimero alpine sleep 3600
docker exec efimero sh -c 'echo "dato valioso de Colpensiones" > /datos.txt'
docker exec efimero cat /datos.txt      # ahí está
docker rm -f efimero
docker run -d --name efimero2 alpine sleep 3600
docker exec efimero2 cat /datos.txt     # ❌ No such file – otra vez (S1)
docker rm -f efimero2
```

1.2 — Ahora el fix: el volumen.

```
docker volume create mis-datos
docker run -d --name convol -v mis-datos:/datos alpine sleep 3600
docker exec convol sh -c 'echo "dato que SOBREVIVE" > /datos/importante.txt'
docker rm -f convol                        # muere el contenedor...
docker run --rm -v mis-datos:/datos alpine cat /datos/importante.txt  # ✅ isigue ahí!
```

1.3 — Conoce tus volúmenes:

```
docker volume ls
docker volume inspect mis-datos      # dónde vive, cuándo nació
```

🔥 **Para la bitácora:** ¿qué diferencia estructural hay entre /datos.txt (1.1) y /datos/importante.txt (1.2)? ¿Dónde vive cada uno?

Parte 2 · El blog inmortal (25 min)

2.1 — La arquitectura de la S3 + UNA línea (el `-v` de la db):

```
docker volume create datos_db

docker run -d --name db --network ean-net \
  -v datos_db:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=Ean2026root \
  -e MYSQL_DATABASE=wordpress \
  -e MYSQL_USER=wp -e MYSQL_PASSWORD=Ean2026wp \
  mysql:8.0

docker logs -f db      # espera el «ready for connections» (la 2ª vez) y Ctrl+C

docker run -d --name blog --network ean-net -p 8080:80 \
  -e WORDPRESS_DB_HOST=db -e WORDPRESS_DB_USER=wp \
  -e WORDPRESS_DB_PASSWORD=Ean2026wp -e WORDPRESS_DB_NAME=wordpress \
  wordpress:6
```

2.2 — Instala el blog (enlace 8080, como en la S3) y publica un post con tu nombre en el título.

2.3 — La prueba de fuego. Mata la base de datos... CON los posts adentro:

```
docker rm -f db      # 💀 (la S3 esto era una tragedia)
```

Recarga el blog: “Error establishing a database connection”. Y ahora, resucítala **montando el mismo volumen**:

```
docker run -d --name db --network ean-net \
  -v datos_db:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=Ean2026root \
  mysql:8.0
```

(Fíjate: ni siquiera repetimos las env de creación — los datos YA existen en el volumen.) Espera ~30 s, recarga el blog → **tu post está intacto.** 🎉

🔥 **Para la bitácora:** captura del post ANTES y DESPUÉS de matar la db. ¿Por qué esta vez sobrevivió?

Parte 3 · Respaldo y restauración PROBADA (20 min)

3.1 — El respaldo (🕒 arranca tu cronómetro cuando llegues al desastre):

```
docker exec db mysqldump -u root -p"Ean2026root" wordpress > respaldo.sql
ls -lh respaldo.sql
head -20 respaldo.sql          # tu blog completo, en texto plano
```

3.2 — El desastre TOTAL. Esta vez no sobrevive nada — ni el volumen:

```
docker rm -f db blog
docker volume rm datos_db      # 💀💀 adiós datos
docker volume ls              # confirmado: no está
```

3.3 — La reconstrucción desde cero (🕒 cronómetro corriendo):

```
docker volume create datos_db

docker run -d --name db --network ean-net \
  -v datos_db:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=Ean2026root \
  -e MYSQL_DATABASE=wordpress \
  -e MYSQL_USER=wp -e MYSQL_PASSWORD=Ean2026wp \
  mysql:8.0

docker logs -f db              # espera el ready... (2ª vez) y Ctrl+C
```

3.4 — La restauración:

```
docker exec -i db mysql -u root -p"Ean2026root" wordpress < respaldo.sql
```

(El `-i` es la clave: alimenta el archivo al contenedor por stdin.)

3.5 — Verifica:

```
docker run -d --name blog --network ean-net -p 8080:80 \
  -e WORDPRESS_DB_HOST=db -e WORDPRESS_DB_USER=wp \
  -e WORDPRESS_DB_PASSWORD=Ean2026wp -e WORDPRESS_DB_NAME=wordpress \
  wordpress:6
```

Recarga el 8080: **tu post volvió de la muerte total.** 🕒 Para el cronómetro: ese es tu **RTO** (tiempo de recuperación).

👉 **Para la bitácora:** tu RTO en minutos. ¿Qué harías para bajarlo? ¿Dónde debería vivir respaldo.sql para sobrevivir a la muerte del HOST completo? (pista: la demo del docente.)

3.6 — Limpieza: `docker rm -f blog db && docker volume rm datos_db mis-datos && docker network rm ean-net`

Checklist de la sesión

- ☐ Comprobé la pérdida sin volumen y la supervivencia con volumen.
- ☐ Mi blog sobrevivió a la muerte de la db gracias a datos_db.
- ☐ Hice mysqldump y revisé el contenido del respaldo.
- ☐ Simulé el desastre total (contenedor + volumen).
- ☐ Restauré desde el .sql y verifiqué mi post de vuelta.
- ☐ Cronometré mi RTO.

Entregable de la S4

1. Capturas del ciclo: post → db muerta → post de vuelta (Parte 2) → desastre total → restauración (Parte 3).
2. Tu **RTO cronometrado** y las reflexiones de la bitácora.
3. 🚀 **Y arranca tu mini-proyecto:** ver `mini-proyecto.md` en el repositorio — la Fase 1 (tu app con red + volumen + respaldo probado) se trae lista a la S5.

Próxima sesión (cierre): portabilidad local → nube, introducción a la orquestación con Kubernetes, y la **demo final de tu mini-proyecto.** 🚀